

UNIVERSAL MINER PRO

ULTIMATE EDITION v4.0

Full Technical Documentation · Architecture Reference · User Guide

WALLET ENFORCED

41-NODE SWARM

OFFLINE-CAPABLE

ARM NATIVE

Francois Nel · AI Technology PTY LTD
South Africa ■■ · Built on Hisense Y82 Pro · Built for Everyone
Generated: 05 June 2026

MIT Licensed · Open Source · No Subscription · No Ads · No Middleman

Table of Contents

1 Project Overview & Purpose

2 Architecture Summary

- 2.1 Module Map
- 2.2 Data Flow
- 2.3 Technology Stack

3 System Requirements

- 3.1 Supported Platforms & Architectures
- 3.2 Python Dependencies
- 3.3 System Build Dependencies

4 Installation Guide

- 4.1 Android / Userland Ubuntu (Recommended)
- 4.2 Android / Termux (Alternative)
- 4.3 Linux Desktop / Server
- 4.4 macOS
- 4.5 Windows (WSL2)
- 4.6 Python-only (No Build)

5 Module Technical Reference

- 5.1 universal_miner.py — Flask Application Core
- 5.2 miner_config.py — Configuration Engine
- 5.3 miner_manager.py — Process Manager
- 5.4 miner_swarm.py — Swarm Engine
- 5.5 ump_hardware_detect.py — Hardware Detection
- 5.6 ump_profit_switcher.py — Profit Switching Daemon
- 5.7 ump_miner_registry.py — Coin & Algorithm Registry
- 5.8 ump_universal_installer.py — Python Installer

6 Wallet System

- 6.1 wallet_core.py — HD Wallet Engine
- 6.2 wallet_db.py — Encrypted Storage
- 6.3 wallet_api.py — Flask Blueprint
- 6.4 wallet_validator.py — Pre-flight Validator
- 6.5 wallet_enforcer.py — Security Enforcer

7 Supported Coins & Algorithms

8 Mining Pool Reference

9 API Endpoint Reference

10 Configuration & Scripts Reference

- 10.1 Gunicorn Configuration
- 10.2 start.sh / stop.sh / restart.sh
- 10.3 wsgi.py

11 Database Schema

12 Security Model

13 Performance & Hardware Benchmarks

14 Troubleshooting Reference

15 Complete User Guide — Step by Step

15.1 First-Time Setup

15.2 Creating Your Wallet

15.3 Configuring a Mining Profile

15.4 Starting & Monitoring Mining

15.5 Reading & Understanding Your Hashrate

15.6 Using the Earnings Calculator

15.7 Managing Multiple Profiles

15.8 Wallet Operations

15.9 Profit Auto-Switching

15.10 Running in Production (Gunicorn)

15.11 Background Operation (screen)

16 Disclaimer & Earnings Transparency

1. Project Overview & Purpose

Universal Miner Pro ULTIMATE Edition v4.0 is a self-contained, open-source cryptocurrency mining management platform designed to run on *any* device — from an old Android smartphone to a high-end multi-GPU server. It is the result of a single developer's vision: to put real mining tools in the hands of anyone who owns a device, regardless of budget or technical expertise.

The project is authored entirely by **Francois Nel** of AI Technology PTY LTD, built and tested on a Hisense Y82 Pro Android phone running Userland Ubuntu (armv7l, 32-bit). Every architectural decision was made with constrained ARM hardware in mind — no f-strings, no walrus operators, Python 3.6+ compatibility throughout, and zero reliance on cloud services or subscription platforms.

Core Philosophy

- Real deterministic software — no stubs, no mocks, no LLM wrappers
- Offline-first: the wallet, validator, and core miner work without internet
- Self-custodial: private keys never leave the device
- Complete ownership of every layer of the stack
- Honest earnings expectations — built-in calculator, no inflated promises
- MIT licensed — free to use, fork, and redistribute

What UMP Provides

UMP bundles everything required for end-to-end cryptocurrency mining into a single deployable directory. There is no separate backend service to configure, no Docker container to maintain, and no cloud account required. The package contains:

- A Flask web application serving a browser-based dashboard on port 8080
- A full HD blockchain wallet (BIP39/BIP44) for BTC, LTC, DASH, ETH, and XMR
- A 41-node swarm engine that spawns up to 40 parallel miner subprocesses
- Dual-algorithm support: RandomX (XMRig) and X11 (cpuminer-multi)
- Automatic profit switching every 5 minutes via CoinGecko price feeds
- Pre-flight credential validation before any mining starts
- A complete REST API with 50+ endpoints
- Production WSGI server via Gunicorn with SSE-compatible threading
- A fully automated install script that builds XMRig and cpuminer-multi from source

2. Architecture Summary

UMP follows a modular, single-process Flask architecture where all subsystems share a single SQLite database (WAL mode). The Flask application registers multiple Blueprints, each owning a distinct concern. Background threads handle long-running tasks (swarm nodes, profit switcher, balance refreshes) without blocking the HTTP server.

2.1 Module Map

Module	Role	Key Responsibilities
universal_miner.py	Flask App Core	App factory, blueprint registration, auto-resume, health check
miner_config.py	Config Engine	Coins/pools DB, profile CRUD, miner command builder, dual-algo config
miner_manager.py	Process Manager	XMRig subprocess spawn, stdout parser, stats aggregation
miner_swarm.py	Swarm Engine	41-node orchestration, price feed, SSE stream, hardware monitor
ump_hardware_detect.py	HW Detection	CPU/GPU detection, platform ID, arch sniffing, capability flags
ump_profit_switcher.py	Profit Daemon	APScheduler job, CoinGecko polling, algo/coin switch logic
ump_miner_registry.py	Coin Registry	Full coin/algo/pool/command mapping for 30+ coins
ump_universal_installer.py	Python Installer	Python-driven alternative installer for advanced setups
wallet_core.py	HD Wallet Engine	BIP39/BIP44/BIP32, address derivation, tx signing, broadcast
wallet_db.py	Wallet Storage	AES-256-GCM encrypted keystore, balance cache, TX history
wallet_api.py	Wallet Blueprint	Flask routes for all wallet operations, Binance integration
wallet_validator.py	Pre-flight Validator	Address regex, API key checks, pool connectivity tests
wallet_enforcer.py	Security Enforcer	Forces all miner payouts to internal wallet addresses
gunicorn_config.py	WSGI Config	ARM-tuned Gunicorn: 1 worker, 4 threads, gthread, SSE-safe
wsgi.py	WSGI Entry	Gunicorn entry point, auto-resume daemon thread startup

2.2 Data Flow

The data flow through UMP follows a clear layered pattern:

Browser / HTTP Client → Connects to http://DEVICE_IP:8080 — served by Gunicorn (production) or Flask dev server

Flask Blueprints → Route requests to miner_swarm (mining ops), wallet_api (wallet ops), or core app routes

miner_config.py → Loads profile from miner.db, applies wallet_enforcer to overwrite any address fields

wallet_enforcer.py → Queries wallet.db for the active wallet's coin addresses, substitutes them into the profile

miner_manager / swarm → Builds XMRig or cpuminer-multi command string, spawns subprocess, reads stdout

SQLite miner.db → Stores profiles, sessions, balance cache, price cache, mining credits

SQLite wallet.db → Stores encrypted mnemonic, derived addresses, private keys (AES-256-GCM), TX history

External: Pool Stratum → XMRig/cpuminer connects outbound to pool stratum TCP endpoint

External: CoinGecko API → Profit switcher and balance refresh fetch live prices every 2 min

External: Blockchain APIs → wallet_core fetches live balances from public block explorers

2.3 Technology Stack

Layer	Technology	Version / Notes
Web Framework	Flask	>= 2.0, Blueprint architecture
WSGI Server	Gunicorn	>= 20.1, gthread worker, ARM-tuned
Database	SQLite (WAL mode)	Two databases: miner.db + wallet.db
CPU Miner (RandomX)	XMRig	Built from source per architecture
CPU Miner (X11)	cpuminer-multi	Built from source per architecture
Scheduling	APScheduler	>= 3.10, background scheduler
Encryption	cryptography (AESGCM)	>= 41.0.0, AES-256-GCM + PBKDF2
System Monitor	psutil	>= 5.9
HTTP Requests	requests	>= 2.28
PDF Generation	reportlab	>= 4.0 (for internal PDF reports)
Real-time Push	SSE (Server-Sent Events)	Native Flask streaming response
HD Wallet Crypto	Pure Python + cryptography	BIP39/BIP32/BIP44, secp256k1
Price Feed	CoinGecko API v3	Free tier, no key required
Frontend	Vanilla HTML/JS/CSS	Tailwind-inspired, inline templates
Language	Python 3.6+	No f-strings, no walrus — ARM 32-bit safe

3. System Requirements

3.1 Supported Platforms & Architectures

Platform	Architecture	Status	Notes
Android / Userland Ubuntu	armv7l	✓ Recommended	Full Ubuntu env, best ARM performance
Android / Userland Ubuntu	aarch64	✓ Recommended	64-bit ARM, full feature set
Android / Termux	armv7l	✓ Supported	Install from F-Droid only, not Play Store
Android / Termux	aarch64	✓ Supported	Good alternative to Userland
Linux Desktop / Server	x86_64	✓ Full Support	CPU + GPU mining, all features
Linux Server	arm64	✓ Supported	Headless-friendly
Raspberry Pi	armv7l / arm64	✓ Functional	Low hashrate but fully operational
macOS	x86_64	✓ Supported	Homebrew required for build deps
macOS	Apple M-chip	✓ Supported	Rosetta or native ARM build
Windows	x86_64	✓ Via WSL2	WSL2 Ubuntu recommended

3.2 Python Dependencies

Package	Min Version	Purpose
flask	2.0	Web framework, routing, blueprints, SSE
requests	2.28	HTTP client for CoinGecko, blockchain APIs, pool ping
psutil	5.9	CPU usage, memory, temperature monitoring
apscheduler	3.10	Background scheduler for profit switcher
gunicorn	20.1	Production WSGI server
cryptography	41.0.0	AES-256-GCM encryption, PBKDF2, Ed25519
reportlab	4.0	PDF generation for internal reports

3.3 System Build Dependencies

These are installed automatically by the install script. Listed for manual installations:

Package	Platform	Purpose
build-essential / gcc	Linux/Termux	C/C++ compiler for XMRig and cpuminer-multi
cmake	All	Build system for XMRig (3.5+ required)
git	All	Clone XMRig and cpuminer-multi source repositories

libssl-dev / openssl	All	TLS support for encrypted pool connections
libuv1-dev	Linux	Async I/O library required by XMRig
libhwloc-dev	x86_64/aarch64	CPU topology detection (skipped on armv7l)
libjansson-dev	Linux	JSON library for cpuminer-multi
libcurl4-openssl-dev	Linux	HTTP client for cpuminer-multi
automake / autoconf / libtool	Linux	Build tools for cpuminer-multi autoconf chain
libgmp-dev	Linux	Arbitrary precision math for crypto
screen	All	Background session management
python3-dev	Linux	Python C headers for native extensions

RAM requirement: Minimum 512 MB free RAM for XMRig build. On armv7l with limited RAM, create a 1 GB swapfile before building.

Disk requirement: Minimum 500 MB free disk space for build artifacts and source trees.

4. Installation Guide

The install script (`install_universal_miner.sh`) is a 424-line Bash script that performs 9 sequential steps: platform detection, system package installation, CMake verification/build, XMRig build from source, cpuminer-multi build from source, Python package installation, directory scaffolding, startup script generation, and final verification. On armv7l the full build takes 15–25 minutes. Keep the screen on and device plugged in.

Install Script Steps (Automated)

Step 1/9 — Platform & Architecture Detection

Reads `uname -m`, checks `/data/data/com.termux`, `/proc/version` for Userland markers. Sets ARCH, PLATFORM, PKG_MGR, CPU_COUNT, REC_THREADS.

Step 2/9 — System Package Installation

Runs `apt-get` or `pkg` to install: build-essential, git, wget, curl, libssl-dev, libuv1-dev, automake, autoconf, libtool, pkg-config, libjansson-dev, libcurl4-openssl-dev, python3, python3-pip, libgmp-dev, screen. Skips `libhwloc-dev` on armv7l.

Step 3/9 — CMake Verification

Checks if `cmake` ≥ 3.5 is available. If not, downloads and builds CMake from source. Required for XMRig build system.

Step 4/9 — XMRig Build

Clones `xmrigh/xmrigh` from GitHub. Configures with `cmake -DWITH_HWLOC=OFF` on armv7l. Builds with `make -j(CPU_COUNT)`. Copies binary to `bin/xmrigh`.

Step 5/9 — cpuminer-multi Build

Clones `tpuvot/cpuminer-multi` from GitHub. Runs `./autogen.sh` then `./configure` with `--disable-assembly` on ARM. Builds with `make -j(CPU_COUNT)`. Copies binary to `bin/cpuminer-multi`.

Step 6/9 — Python Packages

Runs `pip3 install` with `--break-system-packages` for all packages in `requirements.txt`.

Step 7/9 — Directory Scaffolding

Creates `bin/`, `logs/` directories. Writes `logs/.gitkeep` and `bin/README.txt`.

Step 8/9 — Startup Script Generation

Writes `start.sh`, `start_screen.sh`, `stop.sh`, `status.sh` with correct permissions (`chmod +x`).

Step 9/9 — Verification

Checks that `bin/xmrigh` and `bin/cpuminer-multi` exist and are executable. Reports ERRORS count. Prints dashboard URL.

4.1 Android / Userland Ubuntu (Recommended)

Userland provides a full Ubuntu Linux environment inside Android via proot. This is the recommended path for Android users as it offers better package support, more stable builds, and better performance than Termux for UMP.

```
# 1. Install Userland from the Google Play Store
# 2. Launch Ubuntu session in Userland
# 3. Update and install build dependencies:
apt-get update && apt-get upgrade -y
apt-get install -y git python3 python3-pip cmake make gcc \
```

```

libssl-dev libuv1-dev build-essential

# 4. Clone the repository
git clone https://github.com/aitechnologyptyltd-cyber/UNIVERSAL-CRYPTO-MINER.git
cd UNIVERSAL-CRYPTO-MINER

# 5. Run the installer (takes 15-25 min on ARM)
chmod +x install_universal_miner.sh
./install_universal_miner.sh

# 6. Start the dashboard
./start.sh

# 7. Open in browser
http://localhost:8080 (from phone)
http://YOUR_PHONE_IP:8080 (from PC on same WiFi)

```

IMPORTANT — Swap for armv7l: If the build fails with an out-of-memory error, create a swapfile first:

```

fallocate -l 1G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile

```

4.2 Android / Termux (Alternative)

Critical: Install Termux from F-Droid only. The Play Store version is outdated and will cause package conflicts.

```

pkg update && pkg upgrade -y
pkg install git python cmake make gcc openssl libuv automake autoconf
git clone https://github.com/aitechnologyptyltd-cyber/UNIVERSAL-CRYPTO-MINER.git
cd UNIVERSAL-CRYPTO-MINER
chmod +x install_universal_miner.sh
./install_universal_miner.sh
./start.sh

```

4.3 Linux Desktop / Server

```

git clone https://github.com/aitechnologyptyltd-cyber/UNIVERSAL-CRYPTO-MINER.git
cd UNIVERSAL-CRYPTO-MINER
chmod +x install_universal_miner.sh
./install_universal_miner.sh

./start.sh # Foreground (see logs directly)
./start_screen.sh # Background (screen session – survives terminal close)
./start_gunicorn.sh # Production (Gunicorn WSGI server – recommended for server)

```

4.4 macOS

```

# Install Homebrew if not present
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install build dependencies
brew install cmake openssl libuv automake autoconf pkg-config

git clone https://github.com/aitechnologyptyltd-cyber/UNIVERSAL-CRYPTO-MINER.git
cd UNIVERSAL-CRYPTO-MINER
chmod +x install_universal_miner.sh
./install_universal_miner.sh
./start.sh

```

4.5 Windows (WSL2)

```
# In PowerShell (Administrator):  
wsl --install -d Ubuntu  
  
# Then in WSL2 Ubuntu terminal:  
sudo apt-get update && sudo apt-get install -y git python3 python3-pip cmake make gcc libssl-dev  
libuv1-dev build-essential  
git clone https://github.com/aitechnologyptyltd-cyber/UNIVERSAL-CRYPTO-MINER.git  
cd UNIVERSAL-CRYPTO-MINER  
chmod +x install_universal_miner.sh  
./install_universal_miner.sh  
./start.sh
```

4.6 Python-only (No Build)

Use this if you already have XMRig installed separately, or want to run the dashboard without mining:

```
pip install -r requirements.txt --break-system-packages  
python3 universal_miner.py
```

5. Module Technical Reference

5.1 universal_miner.py — Flask Application Core

This is the main entry point. It creates the Flask application, registers all blueprints, initialises the database schema, starts background services, and runs the dev server or exposes the WSGI app to Gunicorn.

Key responsibilities:

- Flask app factory with explicit Blueprint registration (no dynamic scanning)
- Registers: swarm_bp (miner_swarm), wallet_bp (wallet_api), main routes
- Calls init_db() on startup to create miner.db schema if not present
- auto_resume(): checks for an active mining session in DB on startup and resumes it
- SSE /stream endpoint: yields real-time JSON stats every second
- /health endpoint: returns JSON with version, uptime, platform, arch
- Runs on 0.0.0.0:8080 — accessible from local network

5.2 miner_config.py — Configuration Engine

The configuration backbone of UMP. Contains the full coin registry, pool registry, algorithm configuration, and all profile CRUD operations.

Key constants and structures:

- TOTAL_NODES = 40 (swarm nodes; +1 master = 41 total miner processes)
- DEFAULT_ALGO = 'RANDOMX'
- DEFAULT_THREADS = 2
- ALGOS dict: full config for RANDOMX and X11 including pool URLs, worker format, HR estimates
- COINS dict: 30+ coin definitions with algorithm, pool mappings, payout coin

Critical function — get_miner_cmd(profile, threads, algo_key):

Builds the complete command-line string for XMRig or cpuminer-multi. Critically, it calls enforce_internal_wallet(profile) first, which overwrites any wallet address fields in the profile with addresses from the internal HD wallet. This is the primary security enforcement point — no external address can be passed to the miner binary.

- Constructs: binary_path -o pool_url -u username -p password --algo ALGO --threads N
- Appends --tls and -k flags for encrypted pool connections (RandomX)
- Returns (cmd_list, warnings[]) tuple — warnings include enforcer substitution notices

5.3 miner_manager.py — Process Manager

Manages the primary XMRig or cpuminer-multi subprocess. Handles process lifecycle, stdout parsing, stats aggregation, and session recording.

- start_miner(profile, threads): spawns subprocess.Popen with stdout/stderr pipes
- stop_miner(): sends SIGTERM, waits 3 seconds, sends SIGKILL if still alive
- Reader thread: reads stdout line-by-line in background thread, calls _parse_line()
- _parse_xmrig(): regex parses 'speed N H/s' lines from XMRig stdout

- `_parse_cpuminer()`: regex parses cpuminer-multi hashrate output format
- Records mining sessions to sessions table in miner.db (start time, hashrate, coin, pool)
- Exposes `get_stats()` dict: {running, hashrate, hr_unit, accepted, rejected, uptime}

5.4 miner_swarm.py — Swarm Engine

The most complex module. Implements the SwarmEngine (orchestrator) and SwarmNode (single miner subprocess) classes. Manages up to 40 parallel miner instances simultaneously, aggregates their hashrates, and serves the price feed and hardware monitor.

SwarmNode class:

- `node_id`: integer 0–39
- Spawns its own XMRig or cpuminer-multi subprocess with `NODE_THREADS=1` thread
- Runs a reader thread parsing stdout continuously
- Reports hashrate, accepted/rejected shares, alive status back to SwarmEngine via `on_update` callback
- Nodes are staggered by `STAGGER_SEC=0.25s` on start to avoid simultaneous process spawning

SwarmEngine class:

- `start_swarm(profile, n_nodes, algo_key)`: starts up to `n_nodes` SwarmNodes
- `stop_swarm()`: stops all nodes cleanly
- `get_swarm_status()`: returns aggregate dict: total_hashrate, node_count, alive_count, per-node stats
- Price feed: polls CoinGecko `/simple/price` every 120s, caches in memory and miner.db
- Hardware monitor: psutil-based CPU%, memory%, temperature polling every 10s
- Snapshot history: records hashrate snapshots every 60s for dashboard chart
- Flask Blueprint (`swarm_bp`): `/api/swarm/status`, `/api/swarm/prices`, `/api/swarm/hardware`, `/stream SSE`

5.5 ump_hardware_detect.py — Hardware Detection

Standalone hardware detection module used at startup and via `/api/sysinfo` endpoint. Provides comprehensive information about the host device.

Function	Returns
<code>detect_platform()</code>	One of: termux, userland_ubuntu, linux, macos, windows
<code>detect_arch()</code>	One of: armv7l, aarch64, x86_64, x86
<code>detect_cpu_info()</code>	Dict: model, cores, threads, freq_mhz, cache_kb, vendor
<code>detect_gpu_info()</code>	List of GPU dicts: name, vram_mb, driver, type (NVIDIA/AMD/Intel)
<code>detect_capability()</code>	One of: CPU_ONLY, GPU_ONLY, CPU_GPU
<code>get_full_report()</code>	Complete dict combining all above + OS details, memory, disk
<code>recommend_threads()</code>	Integer: $\max(1, \text{cpu_cores} - 1)$ — leaves one core for OS

5.6 ump_profit_switcher.py — Profit Switching Daemon

An APScheduler-driven background daemon that automatically switches between coins and algorithms to maximise mining profitability.

- Runs every SWITCH_INTERVAL_SEC = 300 seconds (5 minutes)
- Fetches prices for 25+ coins from CoinGecko API (/simple/price endpoint)
- Scores each minable coin: $\text{score} = (\text{hashrate} * \text{price}) / \text{network_difficulty_factor}$
- Only switches if new coin earns MIN_SWITCH_GAIN_PCT = 5% more than current
- Logs every switch decision to logs/profit_switch.log with old/new coin, score delta
- Updates active profile in miner.db, signals miner_manager to restart with new config
- PRICE_CACHE_TTL = 120 seconds — prices older than 2 min trigger a fresh fetch
- Gracefully handles CoinGecko rate limits with exponential backoff
- CoinGecko IDs tracked: XMR, ZEPH, SAL, DOGE, ADA, SOL, BNB, TRX, XRP, RVN, ETC, ERG, KAS, ALPH, BTC, LTC, BCH, RTM, WOW, and more

5.7 ump_miner_registry.py — Coin & Algorithm Registry

A comprehensive static registry mapping every supported coin to its algorithm, binary, pool configurations, and miner command patterns. Used by miner_config.py to build correct commands and by the Setup UI to populate dropdowns.

- ALGOS dict: 10+ algorithm definitions (RANDOMX, X11, X11_DIRECT, KAWPOW, ETCHASH, AUTOLYKOS2, SHA3X, HEAVYHASH, EQUIHASH, BEAMHASH)
- COINS dict: 30+ coin entries each with algo_key, display_name, ticker, capability (CPU/GPU/ASIC), pool suggestions
- POOL_URLS dict: per-coin stratum URLs for Kryptex, ViaBTC, MoneroOcean, Unmineable, MPH, Binance
- build_command(coin, pool, wallet, worker, threads, referral): generates full miner command for any combination
- get_recommended_config(arch, capability): returns best coin/algo for the detected hardware

5.8 ump_universal_installer.py — Python Installer

A Python-based alternative to install_universal_miner.sh for environments where Bash is unavailable or restricted. Performs the same 9 installation steps using subprocess calls and Python's built-in OS libraries.

- Detects platform and architecture using Python platform and os modules
- Installs system packages via subprocess.run(['apt-get', 'install', ...])
- Clones and builds XMRig and cpuminer-multi using subprocess
- Installs Python packages via pip subprocess
- Provides detailed progress reporting to stdout

6. Wallet System

UMP includes a complete, production-grade, self-custodial HD (Hierarchical Deterministic) blockchain wallet. It is implemented from scratch in pure Python using only the standard library plus the *cryptography* package. No external wallet libraries (bitcoinlib, web3.py, etc.) are required. All private keys are encrypted with AES-256-GCM and stored exclusively on the local device.

6.1 wallet_core.py — HD Wallet Engine

BIP39 — Mnemonic Generation:

Generates cryptographically secure 12-word mnemonics using the official BIP39 English wordlist (2048 words). The wordlist is cached locally at `bip39_english.txt`; on first run it is downloaded from the official Trezor python-mnemonic repository.

- Entropy: 128 bits via `secrets.token_bytes(16)` — cryptographically secure PRNG
- Checksum: SHA-256 of entropy, append 4 bits
- Encoding: 132 bits mapped to 12 words from the 2048-word BIP39 list
- Seed derivation: PBKDF2-HMAC-SHA512, 2048 iterations, with optional passphrase

BIP32 — HD Key Derivation:

- Master key: HMAC-SHA512 of seed with key 'Bitcoin seed'
- Child key derivation: HMAC-SHA512 with hardened and normal child indices
- Secp256k1 EC operations: implemented in pure Python (no external secp256k1 lib)
- Supports full BIP32 path traversal: `m/purpose'/coin_type'/account'/change/index`

BIP44 — Multi-Account Derivation Paths:

Each coin derives its address from a standard BIP44 path:

Coin	BIP44 Path	Address Format	Notes
BTC	<code>m/44'/0'/0'/0/0</code>	P2PKH (1...) + Bech32 (bc1...)	Also derives SegWit address
LTC	<code>m/44'/2'/0'/0/0</code>	P2PKH (L/M...) + Bech32 (ltc1...)	Legacy and native SegWit
DASH	<code>m/44'/5'/0'/0/0</code>	P2PKH (X...)	DASH standard P2PKH
ETH	<code>m/44'/60'/0'/0/0</code>	EIP-55 checksummed 0x...	Keccak-256 based
XMR	<code>m/44'/128'/0'/0/0</code>	95-char base58 address	Ed25519/Curve25519 keys

Address Generation (per coin):

- BTC P2PKH: secp256k1 pubkey → SHA-256 → RIPEMD-160 → Base58Check with version 0x00
- BTC Bech32 (bc1): SegWit v0 witness program → Bech32 encoding
- ETH: secp256k1 pubkey → Keccak-256 → last 20 bytes → EIP-55 checksum
- DASH: same as BTC P2PKH but with version byte 0x4C
- LTC: version byte 0x30 (L addresses) or 0x32 (M addresses)
- XMR: Ed25519 key pair derivation; spend key + view key → 95-char Monero address

Balance Fetching:

- BTC: Blockstream.info API (/api/address/{addr})
- LTC: Blockchair API (/litecoin/dashboards/address/{addr})
- DASH: Blockcypher API (/v1/dash/main/addrs/{addr}/balance)
- ETH: Etherscan API (/api?module=account&action;=balance)
- XMR: MoneroWorld API or local view-key scan
- All fetches are non-blocking; results cached in wallet.db for 5 minutes

Transaction Signing & Broadcast:

- BTC: UTXO-based, P2PKH signing with DER-encoded secp256k1 signatures
- Builds raw transaction hex, broadcasts via Blockstream.info /api/tx POST
- LTC/DASH: same pattern as BTC with coin-specific version bytes and broadcast APIs
- ETH: EIP-155 signed transactions via eth_sendRawTransaction JSON-RPC

6.2 wallet_db.py — Encrypted Storage

Encryption scheme:

- Key derivation: PBKDF2-HMAC-SHA256, 200,000 iterations, 32-byte output — NIST SP 800-132 compliant
- Encryption: AES-256-GCM (authenticated encryption with associated data)
- Nonce: 12 bytes from secrets.token_bytes() — never reused
- Salt: 32 bytes from secrets.token_bytes() — unique per encryption operation
- Storage format: hex(salt[32] + nonce[12] + ciphertext + GCM tag[16])
- A wrong password raises ValueError from AESGCM.decrypt — no oracle attack possible

Database tables in wallet.db:

Table	Contents
wallet_keystore	wallet_name, encrypted_mnemonic (AES-256-GCM hex), salt_hint, timestamps
wallet_coins	wallet_name, coin, address, address_legacy, address_bech32, encrypted_privkey
wallet_session	Key-value store for active wallet name across Flask sessions
balance_cache	wallet_name, coin, balance, usd_value, price, timestamp (5 min TTL)
price_cache	coin, price_usd, timestamp
transactions	txid, wallet_name, coin, from_addr, to_addr, amount, fee, status, timestamp
mining_credits	wallet_name, coin, amount, source (pool/profile), timestamp
address_book	wallet_name, label, coin, address, notes, timestamp

6.3 wallet_api.py — Flask Blueprint

Registered as the 'wallet' Blueprint with url_prefix='/wallet'. Provides all wallet operations as REST endpoints plus the wallet UI templates.

Method	Endpoint	Description
GET	/wallet/	Wallet dashboard UI (wallet_ui.html)

POST	/wallet/api/create	Create new HD wallet — returns mnemonic (show once only!)
POST	/wallet/api/restore	Restore wallet from BIP39 mnemonic
GET	/wallet/api/addresses	List all derived addresses for active wallet
GET	/wallet/api/balances	Live balances — triggers background refresh
GET	/wallet/api/portfolio	Portfolio summary: total USD value, per-coin breakdown
POST	/wallet/api/transfer	Sign and broadcast a transaction
GET	/wallet/api/transactions	Transaction history for active wallet
GET	/wallet/api/mining_credits	Mining payout credits attributed to wallet
POST	/wallet/api/address_book/add	Add address to address book
GET	/wallet/api/address_book	List saved addresses
DELETE	/wallet/api/address_book/<id>	Delete saved address
GET	/wallet/api/binance/deposit	Fetch Binance deposit address for coin
POST	/wallet/api/binance/withdraw	Initiate Binance withdrawal
GET	/wallet/api/wallets	List all wallet names in keystore
POST	/wallet/api/switch	Switch active wallet by name
DELETE	/wallet/api/delete/<name>	Delete wallet from keystore
GET	/wallet/api/export	Export wallet addresses as JSON (no private keys)

6.4 wallet_validator.py — Pre-flight Validator

Runs automatically before any mining session starts. Validates all configured credentials without blocking the app startup. Results are surfaced via the /api/validate endpoint and displayed as inline warnings on the dashboard.

Validation Check	Method	Coins / Pools
XMR address format	Regex: ^4[A-Za-z0-9]{94}\$ or ^8...	XMR — standard and subaddress
BTC address format	Regex: P2PKH ^[13]... or Bech32 ^bc1...	BTC, Kryptex payout
DASH address format	Regex: ^[X7][a-km-zA-HJ-NP-Z1-9]{33}\$	DASH, Binance Pool
LTC address format	Regex: ^[LM3]... or ^ltc1...	LTC addresses
BCH address format	Regex: cashaddr or legacy	BCH
ETH address format	Regex: ^0x[0-9a-fA-F]{40}\$	ETH
Binance API key format	Regex: ^[A-Za-z0-9]{64}\$	Binance Pool, MPH
Worker name validation	Regex: alphanumeric + _ . -	All pools
Unmineable format	COIN:ADDRESS#REFCODE pattern	Unmineable
Pool URL connectivity	TCP socket connect test	All pools
Binance API validity	Signed GET /api/v3/account	Binance integration

MPH API key	GET https://miningpoolhub.com/...	Mining Pool Hub
Thread count	1 <= threads <= cpu_count	All configs

6.5 wallet_enforcer.py — Security Enforcer

This is the critical security module. It intercepts every call to `get_miner_cmd()` and ensures that all wallet address fields in the profile are overwritten with addresses from the internal HD wallet. An external address can NEVER reach the miner binary through the normal code path.

- `enforce_internal_wallet(profile)`: queries `wallet.db` for active wallet addresses, substitutes into profile
- Handles `WALLET_COIN_ALIAS`: ZEPH→XMR, SAL→XMR, ETHW→ETH, BCH→BTC
- `POOL_PAYOUT_COIN` mapping handles pools that pay in a different coin than mined
- Returns `(modified_profile, warnings[])` — warnings are logged and shown on dashboard
- `require_internal_wallet()`: raises if no wallet has been created yet
- If `wallet.db` does not exist, the enforcer returns `(None, None)` and `get_miner_cmd()` aborts
- If a user manually edits `miner.db` to insert an external address, it is overwritten on next start

7. Supported Coins & Algorithms

Coin	Ticker	Algorithm	Capability	Payout Via
Monero	XMR	RandomX (rx/0)	CPU	Direct / Kryptex / MoneroOcean
Zephyr Protocol	ZEPH	RandomX	CPU	Kryptex (XMR address used)
Salvium	SAL	RandomX	CPU	Kryptex (XMR address used)
Bitcoin	BTC	SHA-256	ASIC	Kryptex (auto-convert), ViaBTC
Litecoin	LTC	Scrypt	ASIC/CPU	Kryptex, ViaBTC
Ethereum Classic	ETC	Etchash	GPU	Kryptex, ViaBTC
Dogecoin	DOGE	Scrypt (proxied)	CPU via Unmineable	Unmineable
Cardano	ADA	RandomX (proxied)	CPU via Unmineable	Unmineable
Solana	SOL	RandomX (proxied)	CPU via Unmineable	Unmineable
Dash	DASH	X11	CPU	Binance Pool, ViaBTC
Ravencoin	RVN	KawPow	GPU	Kryptex
Ergo	ERG	Autolykos2	GPU	Kryptex
Kaspa	KAS	HeavyHash	GPU/ASIC	Kryptex, ViaBTC
Alephium	ALPH	SHA3x	GPU	Kryptex
Zcash	ZEC	Equihash	ASIC/GPU	Kryptex, ViaBTC
Ethereum PoW	ETHW	Ethash	GPU	Kryptex (ETH address)
Bitcoin Cash	BCH	SHA-256	ASIC	ViaBTC
Raptoreum	RTM	GhostRider	CPU	Direct
Wownero	WOW	RandomWOW	CPU	Direct
XRP (proxied)	XRP	RandomX proxy	CPU via Unmineable	Unmineable
BNB (proxied)	BNB	RandomX proxy	CPU via Unmineable	Unmineable
MATIC (proxied)	MATIC	RandomX proxy	CPU via Unmineable	Unmineable
Polkadot (proxied)	DOT	RandomX proxy	CPU via Unmineable	Unmineable

Note on Unmineable: When using Unmineable, you physically mine using RandomX (XMRig) but Unmineable converts your hashrate and pays you out in whichever coin you specify — DOGE, ADA, SOL, XRP, BNB, SHIB, and 100+ other coins. The miner user field format is COIN:WALLET_ADDRESS#REFERRAL_CODE.

8. Mining Pool Reference

Kryptex Pool

Parameter	Value
Pool URL	pool.kryptex.com
Protocol	Stratum TCP + TLS
User format	YOUR_KRYPTEX_EMAIL.WORKER_NAME
Password	x
XMR URL	xmr.kryptex.network:8029
ZEPH URL	zeph.kryptex.network:8029
BTC URL	btc.kryptex.network:3333
LTC URL	ltc.kryptex.network:3333
KAS URL	kas.kryptex.network:3333
ZEC URL	zec.kryptex.network:7042
ETC URL	etc.kryptex.network:8888
RVN URL	rvn.kryptex.network:3333
ERG URL	erg.kryptex.network:3333
ALPH URL	alph.kryptex.network:3333
Payout	Auto-converts all mined coins to BTC equivalent
Min payout	0.0005 BTC
Notes	Register at kryptex.com first. Use email as username.

ViaBTC

Parameter	Value
Pool URL	pool.viabtc.com
Protocol	Stratum TCP
User format	VIABTC_USERNAME.WORKER_NAME
Password	123
Payout method	PPS+ (pay-per-share-plus)
Founded	2016 — one of the oldest pools
Coins	BTC, BCH, LTC, ETH, ETC, ZEC, DASH, XMR, KAS
Notes	Register at viabtc.com. Reliable PPS+ payouts.

Unmineable

Parameter	Value
Pool URL	unmineable.com
Protocol	Stratum TCP / SSL
TCP URL	rx.unmineable.com:3333
SSL URL	rx.unmineable.com:443
User format	COIN:YOUR_WALLET_ADDRESS#REF_CODE
Password	WORKER_NAME
Payout coins	100+ coins: BTC, ETH, DOGE, ADA, SOL, SHIB, XRP, BNB, ...
Min payout	Varies by coin — see unmineable.com
Notes	Mine with RandomX CPU, receive any coin. No registration needed.

MoneroOcean

Parameter	Value
Pool URL	moneroocean.stream
TCP URL	gulf.moneroocean.stream:10128
SSL URL	gulf.moneroocean.stream:20128
User	YOUR_XMR_WALLET_ADDRESS (no registration)
Password	WORKER_NAME:your@email.com
Feature	Auto-switches RandomX algorithm variant for max XMR profit
Payout coin	XMR always
Notes	Best for XMR mining. No registration. Highest algo optimization.

Mining Pool Hub

Parameter	Value
Pool URL	miningpoolhub.com
User format	MPH_USERNAME.WORKER_NAME
Password	x
Feature	Auto-switching multi-algo pool
API key	Required — create in account settings at miningpoolhub.com
Notes	Registration required. Must create API key for UMP integration.

Binance Pool

Parameter	Value
Pool URL	pool.binance.com
Stratum URL	stratum+tcp://bs.poolbinance.com:1800
User format	BINANCE_USERNAME.WORKER_NAME
Password	x
Algorithm	X11 (DASH mining)
Payout	Daily to Binance exchange account
Notes	No minimum payout. Requires active Binance account.

9. API Endpoint Reference

All endpoints return JSON unless otherwise noted. Base URL: `http://DEVICE_IP:8080`

Core Application

Method	Endpoint	Description
GET	/	Dashboard HTML page
GET	/setup	Setup wizard HTML page
GET	/profiles	Profile manager HTML page
GET	/health	Health check — {version, uptime, platform, arch, status}
GET	/stream	SSE stream — live stats pushed every 1 second

Mining Operations

Method	Endpoint	Description
POST	/mine/start	Start mining. Body: {threads: N, profile_id: ID}
POST	/mine/stop	Stop all mining processes
POST	/mine/restart	Stop then start mining
POST	/mine/switch_algo	Switch between RANDOMX and X11 live

Stats & Monitoring

Method	Endpoint	Description
GET	/api/stats	Live miner stats: {running, hashrate, hr_unit, accepted, rejected, uptime}
GET	/api/log?lines=80	Last N lines from active miner log
GET	/api/sessions	Mining session history: [{start, stop, hashrate, coin, pool}]
GET	/api/sysinfo	Platform/arch/CPU/GPU detection results
GET	/api/validate	Pre-flight validation results for all profiles

Profiles & Configuration

Method	Endpoint	Description
GET	/api/profiles	List all profiles
GET	/api/profiles/<id>	Get single profile by ID
POST	/api/profiles	Create profile. Body: full profile JSON
PUT	/api/profiles/<id>	Update profile
DELETE	/api/profiles/<id>	Delete profile

GET	/api/coins	Full coin registry from ump_miner_registry
GET	/api/pools	Full pool registry
GET	/api/suggest_pool_url?coin=XMR&pool=kryptex	Auto-suggest stratum URL
GET	/api/hashrate_calc?hs=500&unit=H%2Fs&coin=XMR	Earnings calculator

Swarm Engine

Method	Endpoint	Description
GET	/api/swarm/status	Swarm status: {total_hashrate, node_count, alive, per_node_stats}
POST	/api/swarm/start	Start swarm with N nodes
POST	/api/swarm/stop	Stop all swarm nodes
GET	/api/swarm/prices	Latest CoinGecko prices for all tracked coins
GET	/api/swarm/hardware	CPU%, memory%, temperature from psutil
GET	/api/swarm/history	Hashrate snapshot history for charts

Wallet (prefix: /wallet)

Method	Endpoint	Description
GET	/wallet/	Wallet UI page
POST	/wallet/api/create	Create HD wallet — returns mnemonic ONCE
POST	/wallet/api/restore	Restore from mnemonic
GET	/wallet/api/addresses	All derived addresses
GET	/wallet/api/balances	Live blockchain balances
GET	/wallet/api/portfolio	Portfolio summary with USD values
POST	/wallet/api/transfer	Sign and broadcast transaction
GET	/wallet/api/transactions	Transaction history
GET	/wallet/api/mining_credits	Mining payout history
POST	/wallet/api/address_book/add	Add to address book
GET	/wallet/api/address_book	List address book
GET	/wallet/api/binance/deposit?coin=BTC	Binance deposit address for coin
POST	/wallet/api/binance/withdraw	Initiate Binance withdrawal
GET	/wallet/api/wallets	List wallet names
POST	/wallet/api/switch	Switch active wallet

10. Configuration & Scripts Reference

10.1 Gunicorn Configuration (gunicorn_config.py)

Parameter	Value	Reason
bind	0.0.0.0:8080	Listen on all interfaces — accessible from LAN
worker_class	gthread	Thread-based workers required for SSE long-lived connections
workers	1	Single worker process — saves RAM on armv7l (3.6 GB total)
threads	4	4 threads per worker: dashboard + SSE + API simultaneously
timeout	0	No timeout — SSE connections are indefinitely long-lived
graceful_timeout	10	10s for workers to finish on graceful restart
keepalive	5	Keep-alive for non-SSE HTTP connections
preload_app	True	Load app before forking workers — saves RAM
pidfile	ump.pid	PID file for stop.sh and restart.sh
accesslog	logs/gunicorn_access.log	HTTP access log
errorlog	logs/gunicorn_error.log	Gunicorn error log
loglevel	info	Log verbosity level
proc_name	ump_miner	Process name shown in ps/top

10.2 Shell Scripts

Script	Purpose	Key Behaviour
start.sh	Start Flask dev server (background)	Kills previous instance by PID file; spawns via nohup; writes PID to ump.pid
start_screen.sh	Start in screen session	Creates named 'ump_miner' screen; reattach with: screen -r ump_miner
start_gunicorn.sh	Start production Gunicorn	Uses gunicorn_config.py; recommended for server deployments
stop.sh	Stop all UMP processes	Reads ump.pid; sends SIGTERM, then SIGKILL if needed; pkill fallback
restart.sh	Restart production server	Calls stop.sh then start_gunicorn.sh sequentially
wsgi.py	Gunicorn WSGI entry	Imports app from universal_miner; starts auto_resume daemon thread

10.3 Log Files

Log File	Contents
logs/ump.log	Main Flask application log — startup, errors, info messages
logs/{PROFILE}_xmrig.log	XMRIg stdout per profile — hashrate lines, accepted shares
logs/{PROFILE}_cpuminer.log	cpuminer-multi stdout per profile — X11/DASH hashrate

logs/profit_switch.log	Every profit switching event — old coin, new coin, score delta
logs/gunicorn_access.log	HTTP access log — all requests with status codes
logs/gunicorn_error.log	Gunicorn worker errors and crash reports
build.log	Full output from install_universal_miner.sh — cmake/compiler messages

11. Database Schema

UMP uses two SQLite databases, both opened in WAL (Write-Ahead Log) mode for concurrent read access without blocking writes.

miner.db — Application Database

profiles

```
id INTEGER PRIMARY KEY AUTOINCREMENT
name TEXT UNIQUE NOT NULL
coin TEXT, pool TEXT, pool_url TEXT
miner_user TEXT, miner_pass TEXT
kryptex_wallet TEXT, viabtc_username TEXT, unmineable_wallet TEXT
mph_username TEXT, mph_api_key TEXT
moneroocean_wallet TEXT
binance_api_key TEXT, binance_api_secret TEXT, binance_username TEXT
xmr_wallet TEXT, btc_wallet TEXT
threads INTEGER DEFAULT 2
algo TEXT DEFAULT 'RANDOMX'
tls INTEGER DEFAULT 1
active INTEGER DEFAULT 0
created_at DATETIME, updated_at DATETIME
```

sessions

```
id INTEGER PRIMARY KEY AUTOINCREMENT
profile_id INTEGER, profile_name TEXT
coin TEXT, pool TEXT, algo TEXT
start_time DATETIME, stop_time DATETIME
peak_hashrate REAL, avg_hashrate REAL, hr_unit TEXT
accepted INTEGER DEFAULT 0, rejected INTEGER DEFAULT 0
duration_sec INTEGER
```

balance_cache (miner.db copy)

```
id INTEGER PRIMARY KEY AUTOINCREMENT
wallet_name TEXT, coin TEXT
balance REAL, usd_value REAL, price_usd REAL
updated_at DATETIME
```

price_cache

```
coin TEXT PRIMARY KEY
price_usd REAL
updated_at DATETIME
```

swarm_snapshots

```
id INTEGER PRIMARY KEY AUTOINCREMENT
timestamp DATETIME
total_hashrate REAL, hr_unit TEXT
alive_nodes INTEGER, total_nodes INTEGER
algo TEXT
```

wallet.db — Wallet Database

wallet_keystore

```
id INTEGER PRIMARY KEY AUTOINCREMENT
wallet_name TEXT UNIQUE NOT NULL
encrypted_mnemonic TEXT NOT NULL -- AES-256-GCM hex blob
salt_hint TEXT DEFAULT '' -- Optional user hint (NOT the salt itself)
created_at DATETIME, updated_at DATETIME
```

wallet_coins

```
id INTEGER PRIMARY KEY AUTOINCREMENT
wallet_name TEXT NOT NULL
coin TEXT NOT NULL -- BTC, LTC, DASH, ETH, XMR
address TEXT NOT NULL -- Primary address (P2PKH or native)
address_legacy TEXT DEFAULT '' -- Legacy format where applicable
address_bech32 TEXT DEFAULT '' -- SegWit bech32 where applicable
encrypted_privkey TEXT -- AES-256-GCM encrypted private key hex
derivation_path TEXT -- BIP44 path used
UNIQUE(wallet_name, coin)
```

wallet_session

```
key TEXT PRIMARY KEY
value TEXT
```

transactions

```
id INTEGER PRIMARY KEY AUTOINCREMENT
txid TEXT, wallet_name TEXT, coin TEXT
from_address TEXT, to_address TEXT
amount REAL, fee REAL, status TEXT
raw_hex TEXT -- Signed transaction hex before broadcast
block_height INTEGER, confirmations INTEGER
created_at DATETIME, confirmed_at DATETIME
```

mining_credits

```
id INTEGER PRIMARY KEY AUTOINCREMENT
wallet_name TEXT, coin TEXT
```

```
amount REAL, source TEXT  
pool TEXT, profile_name TEXT  
credited_at DATETIME
```

address_book

```
id INTEGER PRIMARY KEY AUTOINCREMENT  
wallet_name TEXT, label TEXT  
coin TEXT, address TEXT  
notes TEXT, created_at DATETIME
```

12. Security Model

Private Key Security

- All private keys are encrypted with AES-256-GCM before storage in wallet.db
- Encryption key is derived from the user's passphrase via PBKDF2-HMAC-SHA256 (200,000 iterations)
- Each encryption operation uses a unique 32-byte salt and 12-byte nonce — no two encryptions are identical
- GCM authentication tag (16 bytes) detects any tampering with stored ciphertext
- Private keys are never transmitted over the network — all signing is done locally
- The mnemonic is shown ONCE at creation time — it is never displayed again from the application

Wallet Enforcement

- wallet_enforcer.py intercepts every miner command construction call
- All address fields in a profile are overwritten with internal HD wallet addresses before the command is built
- No external address can reach the XMRig or cpuminer-multi binary through the normal UMP code path
- Pool-specific address fields (unmineable_wallet, kryptex_wallet, etc.) are all enforced
- Enforcement warnings are logged and displayed on the dashboard so the user sees every substitution

API Key Security

- Binance and MPH API keys are stored in miner.db (SQLite, local only)
- UMP only ever requires READ ONLY Binance API permissions — Enable Reading only
- Never enable Trading, Withdrawal, or Futures permissions on keys used with UMP
- XMR view key (if provided for balance monitoring) only allows viewing — the spend key is never requested
- API keys are never logged to log files

Network Security

- The dashboard has no authentication by default — intended for trusted local network use
- Recommendation: add nginx reverse proxy with HTTP Basic Auth if exposing to the internet
- All mining pool connections use TLS where available (--tls flag passed to XMRig)
- Pool connectivity test in wallet_validator uses TCP socket only — no credentials sent during test

Mnemonic Security Rules

These rules are critical. Failure to follow them can result in permanent loss of funds:

- Write your 12-word mnemonic on paper immediately after creation
- Store the paper in a secure physical location — safe, locked drawer, or similar
- NEVER store the mnemonic in a digital file, photo, email, or cloud service
- NEVER share the mnemonic with anyone — not support staff, not the developer, not anyone
- The mnemonic is shown ONCE at creation — if you miss it, you must create a new wallet
- Anyone with your mnemonic can derive all your private keys and drain all your wallets

- Test your mnemonic by restoring it in a separate environment before sending significant funds

13. Performance & Hardware Benchmarks

All hashrate figures below are real observed values from actual hardware testing. Hashrate varies significantly based on CPU model, temperature, memory speed, background load, and thread count. Use the built-in earnings calculator with your actual observed hashrate for accurate projections.

Device / Platform	Algorithm	Threads	Observed Hashrate	Notes
Android / Userland Ubuntu (Hisense Y82 Pro)(DASH)	RandomX (XMR)	3	4–13 kH/s	Real observed — tested by developer
Android / Userland Ubuntu (armv7l)	RandomX (XMR)	3	50–300 H/s	Steady on ARM, varies by chip
Android / Userland Ubuntu (aarch64)	RandomX (XMR)	4	150–500 H/s	Better on 64-bit ARM
Raspberry Pi 4 (aarch64)	RandomX (XMR)	3	200–400 H/s	Runs warm — monitor temperature
Mid-range laptop (x86_64, 4 cores)	X11	4	50–200 kH/s	Depends heavily on CPU model
Mid-range laptop (x86_64, 4 cores)	RandomX (XMR)	4	500–2000 H/s	Intel/AMD Core series
Desktop PC (x86_64, 8 cores)	RandomX (XMR)	8	2000–8000 H/s	Scales linearly with core count
Desktop PC (Ryzen 9 5900X, 12 cores)	RandomX (XMR)	12	8000–13000 H/s	High-end desktop performance
Server (32+ cores)	RandomX (XMR)	32	10000–25000+ H/s	Enterprise / VPS server

Swarm Engine Theoretical Capacity

With TOTAL_NODES = 40 and the master node, UMP can spawn 41 simultaneous miner processes. On low-power ARM devices this is typically limited by available RAM and CPU cores. The actual number of useful nodes equals the number of CPU threads minus 2 (OS overhead). Nodes are configured with NODE_THREADS = 1 each — aggregate hashrate = nodes × single_thread_hashrate.

Device Class	Recommended Nodes	Expected Total Hashrate (RandomX)
Android armv7l (4 cores)	2–3 nodes	100–900 H/s total
Android aarch64 (8 cores)	6 nodes	900–3000 H/s total
Raspberry Pi 4 (4 cores)	3 nodes	600–1200 H/s total
Mid-range laptop (8 cores)	6–7 nodes	3000–14000 H/s total
Desktop PC (16 cores)	14 nodes	14000–112000 H/s total
Server (32+ cores)	30 nodes	30000–240000 H/s total

These are theoretical maximums. Real performance depends on CPU cache size (RandomX is L3-cache intensive), memory bandwidth, and thermal throttling on mobile devices.

14. Troubleshooting Reference

Dashboard won't load in browser

Run `python3 universal_miner.py` directly to see startup errors in terminal. Check that port 8080 is not in use by another process. Check `logs/ump.log` for Flask startup errors.

```
ps aux | grep universal_miner
lsof -i :8080
```

XMRIg not found — 'binary not found' error

Re-run `./install_universal_miner.sh`. Check that `bin/xmrig` exists and is executable. Check `build.log` for cmake or compiler errors during the original build.

```
ls -la bin/
bash install_universal_miner.sh
cat build.log | tail -50
```

Hashrate shows 0.00 H/s after starting

Check the pool URL in Setup — it may be empty or incorrect. Click AUTO-FILL in the Pool tab after selecting your coin. Check the miner log for connection refused or authentication errors.

```
cat logs/{PROFILE}_xmrig.log | tail -30
python3 wallet_validator.py
```

Build fails with out-of-memory error on armv7l

Create a 1 GB swapfile before building. This is essential on devices with limited RAM.

```
fallocate -l 1G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
# Then re-run installer
```

Port 8080 already in use

Either stop the conflicting process or change UMP's port in `universal_miner.py`.

```
# Change port:
# Find: port=8080
# Replace: port=5002
nano universal_miner.py
```

Payout not arriving from pool

Wait 24 hours — all major pools pay daily with minimum thresholds. Run the validator to confirm credentials are correct. Check pool's minimum payout threshold — you may not have reached it yet.

```
python3 wallet_validator.py
```

Swarm nodes keep dying / crashing

Reduce `TOTAL_NODES` in `miner_config.py` for low-RAM devices. Monitor memory with `free -m` during mining. Check logs for OOM killer messages.

```
free -m
grep -i oom /var/log/syslog 2>/dev/null
# Reduce TOTAL_NODES in miner_config.py
```

Phone getting very hot during mining

This is normal — mining is CPU-intensive. Reduce thread count in Setup (Hardware tab) from 3 to 2 or 1. Mining may throttle automatically when the CPU overheats — this lowers hashrate.

```
# Reduce threads in Setup UI or in miner_config.py DEFAULT_THREADS
```

BIP39 wordlist unavailable (wallet creation fails)

The wallet_core module needs to download the BIP39 wordlist on first run. Run the command shown in the error to download it manually.

```
python3 -c "import urllib.request; open('bip39_english.txt', 'w').write(urllib.request.urlopen('https://raw.githubusercontent.com/trezor/python-mnemonic/master/src/mnemonic/wordlist/english.txt').read().decode())"
```

Flask ImportError on startup

Python packages are missing or wrong version. Reinstall all requirements.

```
pip3 install -r requirements.txt --break-system-packages
# or:
pip3 install flask requests psutil apscheduler gunicorn cryptography --break-system-packages
```

Gunicorn SSE stream disconnects

Gunicorn timeout is set to 0 in gunicorn_config.py — it should not timeout. If using nginx, add 'proxy_read_timeout 0;' and 'proxy_buffering off;' to the nginx location block.

```
# nginx config:
location /stream {
    proxy_pass http://127.0.0.1:8080;
    proxy_buffering off;
    proxy_read_timeout 0;
    proxy_cache_bypass $http_upgrade;
}
```

Wallet balances show 0 or Error

Balance fetching requires internet access to blockchain explorer APIs. These are free public APIs that may rate-limit. Wait 5 minutes and try again. Check that the address was derived correctly by comparing with a blockchain explorer.

```
# Check address on public explorer:
# BTC: https://blockstream.info/address/{ADDRESS}
# ETH: https://etherscan.io/address/{ADDRESS}
```

15. Complete User Guide — Step by Step

This section walks through every aspect of using Universal Miner Pro ULTIMATE v4.0 from the very beginning, assuming no prior experience with cryptocurrency mining. Follow each step in order for a successful first setup.

15.1 First-Time Setup

After installation completes and you run `./start.sh`, open your browser and navigate to `http://localhost:8080` (on the device running UMP) or `http://YOUR_DEVICE_IP:8080` from another device on the same WiFi network.

What you will see:

- The Dashboard page — shows mining status, hashrate graph, earnings calculator, and system info
- A navigation bar with: Dashboard | Setup | Profiles | Wallet
- A warning banner if no profile has been configured yet
- The hardware detection panel showing your CPU model, arch, and recommended thread count

First-time checklist:

- 1. Create your HD wallet (Section 15.2) — do this FIRST so the enforcer has an address to use
- 2. Configure a mining profile (Section 15.3) — choose your coin, pool, and credentials
- 3. Run the pre-flight validator — go to Setup → Validate to check all credentials
- 4. Start mining (Section 15.4) — click START MINING on the dashboard
- 5. Check the miner log — your hashrate will appear within 30–60 seconds
- 6. Enter your hashrate in the Earnings Calculator for a real earning estimate

15.2 Creating Your Wallet

The wallet MUST be created before mining starts. The `wallet_enforcer` requires an active wallet to route payout addresses. Without a wallet, the miner will refuse to start.

Step-by-step wallet creation:

Navigate to Wallet: Click 'Wallet' in the navigation bar at the top of the dashboard.

Click 'Create New Wallet': A button labeled 'Create New Wallet' will be visible on the wallet page.

Enter a wallet name: Type a name for your wallet (e.g., 'main', 'phone_wallet'). This is just a label — it doesn't affect the cryptography.

Enter a passphrase: Choose a strong passphrase. This encrypts your private keys with AES-256-GCM. You will need this passphrase every time you unlock the wallet. DO NOT forget it — if you lose it, you cannot access stored keys (but you can still restore from mnemonic).

CRITICAL: Save your mnemonic: The 12-word mnemonic phrase will be shown ONCE. Write it on paper immediately. This phrase is the master backup of ALL your cryptocurrency addresses. Anyone with this phrase can access all your funds.

Confirm mnemonic saved: Tick the confirmation checkbox confirming you have written down the mnemonic.

Click 'Create Wallet': The wallet is created. Your BTC, LTC, DASH, ETH, and XMR addresses are derived and shown.

Note your addresses: Copy your XMR address — you will need it when configuring the MoneroOcean or Kryptex profile.

Restoring a wallet: If you already have a mnemonic from a previous UMP installation or any other BIP44-compatible wallet, click 'Restore Wallet', enter your 12 words, and UMP will re-derive all your addresses. The same addresses will be produced every time from the same mnemonic.

15.3 Configuring a Mining Profile

A profile is a complete mining configuration template. You can have multiple profiles for different coins, pools, or hardware setups, and switch between them instantly.

Navigate to Setup: Click 'Setup' in the navigation bar.

The Setup page has several tabs: General, Pool, Credentials, Hardware, Advanced.

General Tab:

- Profile Name: Give this configuration a descriptive name (e.g., 'XMR_Kryptex_Phone')
- Coin: Select the coin you want to mine from the dropdown (e.g., XMR, DASH, DOGE)
- Pool: Select the mining pool (Kryptex, ViaBTC, Unmineable, MoneroOcean, Binance, MPH, Direct)
- Algorithm: Auto-selected based on coin. You can also manually choose RANDOMX or X11.

Pool Tab:

- Pool URL: Click AUTO-FILL to automatically populate the correct stratum URL for your coin+pool combination
- You can also enter a custom stratum URL manually for any unlisted pool
- TLS: Leave enabled for all pools that support it (reduces hashrate loss from corruption)

Credentials Tab (fill what applies to your chosen pool):

Kryptex: Enter your Kryptex account email. Worker name is optional (any alphanumeric string).

ViaBTC: Enter your ViaBTC username. Worker name identifies this device.

Unmineable: Enter the wallet address for your desired PAYOUT coin (e.g., your DOGE address). Optionally add a referral code after #.

MoneroOcean: Enter your XMR wallet address. No registration needed — the address IS the username.

Mining Pool Hub: Enter your MPH username and API key (create at miningpoolhub.com → Account → API).

Binance Pool: Enter your Binance.com account username. Worker name identifies this rig.

Binance API Keys: For balance monitoring and withdrawal features only. Enable READ ONLY permissions.

XMR Wallet: Your Monero wallet address (starts with 4). Used by wallet_enforcer as the payout address.

BTC Wallet: Your Bitcoin wallet address. Used by Kryptex for BTC-equivalent payouts.

Hardware Tab:

- Thread Count: Number of CPU threads to allocate. The hardware detector suggests `cpu_cores - 1`.
- On Android armv7l: use 2–3 threads maximum to avoid thermal throttling.
- On desktop/server: use all cores minus 1–2 for OS responsiveness.

Save Profile:

Click 'Save Profile' at the bottom of the Setup page. The profile is stored in miner.db. You will be redirected to the Dashboard where your new profile appears in the dropdown.

Running the Validator:

Before starting mining for the first time, click the 'Validate' button in Setup or visit `/api/validate`. The validator checks all your credentials, address formats, worker names, and pool connectivity. Any warnings or errors are shown inline. Fix all errors before proceeding.

15.4 Starting & Monitoring Mining

Starting the miner:

- Go to the Dashboard
- Select your profile from the dropdown at the top
- Click the green 'START MINING' button
- The miner log panel will begin filling with output from XMRig or cpuminer-multi
- Within 30–60 seconds, you will see your per-thread hashrate lines
- The status panel will show: RUNNING, current hashrate, accepted/rejected shares

Monitoring the live dashboard:

- Hashrate graph: updates every second via SSE stream — shows real-time and historical hashrate
- System panel: CPU%, memory%, temperature from psutil (updates every 10s)
- Miner log: scrolling live log of XMRig/cpuminer output — useful for debugging
- Swarm status: if using swarm mode, shows alive node count and per-node hashrates
- Price feed: live CoinGecko prices for tracked coins (updates every 2 min)

Switching algorithms live:

You can switch between RandomX and X11 without stopping mining — click the algorithm toggle button on the Dashboard. The miner will stop, rebuild the command with the new algorithm, and restart automatically.

Stopping the miner:

- Click 'STOP MINING' on the Dashboard, OR
- Run `./stop.sh` from the terminal
- The miner process receives SIGTERM, then SIGKILL if it doesn't exit within 3 seconds
- The mining session is recorded in the sessions table in miner.db

15.5 Reading & Understanding Your Hashrate

Once mining is active, the miner log will show output like this:

```
[2026-05-28 16:07:09] speed 10s/60s/15m 3.19 3.20 3.09 H/s max 3.45 H/s
[2026-05-28 16:07:09] CPU #0: 3.19 kH/s
[2026-05-28 16:07:09] CPU #1: 3.20 kH/s
[2026-05-28 16:07:09] CPU #2: 3.09 kH/s
[2026-05-28 16:07:10] accepted (12/0) diff 12345 (34 ms)
```

- The 'speed' line shows hashrate over 10s / 60s / 15min windows — use the 60s average as your benchmark
- Per-CPU lines show individual thread contribution — add them up for total
- 'accepted (12/0)' = 12 shares accepted, 0 rejected — a rejection rate above 5% indicates connection issues
- H/s = Hashes per second (RandomX). kH/s = kilohashes/s (X11). MH/s = megahashes/s.
- Hashrate will vary as the CPU warms up — it typically stabilises after 5–10 minutes

- On phones: thermal throttling (CPU slowing down when hot) is normal — plug in the charger and use a cooling pad if available

15.6 Using the Earnings Calculator

The Earnings Calculator is on the Dashboard. It gives you a realistic estimate of what your hashrate is worth at current market prices.

- Enter your observed hashrate from the miner log (e.g., 9.48)
- Select the unit: H/s (RandomX), KH/s (X11 on phone), MH/s (X11 on PC), GH/s (ASIC)
- Select the coin you are mining
- The calculator fetches the current coin price from CoinGecko (updates every 2 min)
- Results show: Daily / Weekly / Monthly / Yearly projections in both coin and USD

Important caveat: Cryptocurrency prices are highly volatile. The USD estimates will change daily as prices move. The hashrate number is what your hardware physically produces and stays relatively constant. The fiat value of that hashrate depends entirely on coin prices and network difficulty, both of which change constantly. Always base decisions on the hashrate number first, not the fiat estimate.

15.7 Managing Multiple Profiles

You can create as many profiles as you like — one per coin, one per pool, or one per device.

- Navigate to Profiles (/profiles) to see all saved profiles
- Click any profile to edit it in Setup
- Delete profiles you no longer need
- Export profiles as JSON for backup (from Profiles page)
- On the Dashboard, select a profile from the dropdown — no restart required
- Profile switching stops the current miner and starts a new one with the selected profile's configuration

Recommended profiles to create:

- XMR_Kryptex: XMR mining on Kryptex — stable, auto-converts to BTC
- XMR_MoneroOcean: XMR mining on MoneroOcean — no registration, best XMR rates
- DOGE_Unmineable: Mine RandomX, receive DOGE — great for specific altcoin accumulation
- DASH_Binance: X11 DASH mining paid to Binance account — convenient if you use Binance

15.8 Wallet Operations

Viewing your balances:

- Navigate to Wallet in the navigation bar
- Click 'Refresh Balances' — this triggers a background fetch from blockchain explorer APIs
- Balances are cached for 5 minutes — subsequent refreshes within 5 min show cached data
- Portfolio summary shows total USD value across all coins

Sending a transaction:

- Click 'Send' next to the coin you want to send
- Enter the destination address — double-check it! Blockchain transactions are irreversible.
- Enter the amount

- Enter your wallet passphrase — required to decrypt the private key for signing
- Review the transaction summary including estimated fee
- Click 'Confirm & Send' — the transaction is signed locally and broadcast to the network
- The txid (transaction ID) is saved to your transaction history

Binance integration:

- If you have Binance API keys configured in your profile (READ ONLY), you can fetch your Binance deposit addresses
- This is useful for knowing where to send mined coins to your exchange account
- The withdraw function requires Binance withdrawal permissions — enable only if needed
- Always double-check Binance withdrawal addresses independently before sending

15.9 Profit Auto-Switching

UMP includes an automatic profit switching daemon that runs in the background and switches to the most profitable coin every 5 minutes.

- Enable profit switching in Setup → Advanced → Enable Profit Switching
- The daemon polls CoinGecko every 2 minutes for current prices of 25+ coins
- It scores each minable coin: $\text{score} = (\text{your_hashrate} \times \text{coin_price}) / \text{network_difficulty}$
- It only switches if the new coin would earn at least 5% MORE than the current coin
- Every switch decision is logged to logs/profit_switch.log with before/after details
- You can disable auto-switching and return to manual profile selection at any time
- If CoinGecko is unreachable, the daemon keeps the current coin and retries on the next cycle

15.10 Running in Production (Gunicorn)

For persistent server deployments, use Gunicorn instead of the Flask dev server:

```
# Start with Gunicorn (recommended for servers)
./start_gunicorn.sh

# Or directly:
gunicorn --config gunicorn_config.py wsgi:app

# Stop Gunicorn
./stop.sh

# Restart
./restart.sh
```

Gunicorn advantages over Flask dev server:

- More stable for long-running deployments
- Pre-forks worker process before accepting connections
- gthread worker handles multiple concurrent SSE connections without blocking
- Proper signal handling for graceful restart
- Access and error logs written to logs/ directory

15.11 Background Operation (screen)

To keep UMP running after closing your terminal session, use GNU screen:

```
# Start UMP in a screen session
./start_screen.sh

# Detach from screen: press Ctrl+A then D

# List screen sessions
screen -ls

# Reattach to UMP session
screen -r ump_miner

# Kill the session (stop UMP)
./stop.sh
screen -X -S ump_miner quit
```

On Android Userland: when you close the Userland app, the screen session persists and UMP continues mining in the background.

16. Disclaimer & Earnings Transparency

Universal Miner Pro ULTIMATE v4.0 is open-source software provided under the MIT License. The developer makes no guarantee of earnings, profitability, or return on investment. Cryptocurrency mining profitability is determined by factors entirely outside the control of this software.

Factors That Determine Your Earnings

- Your hardware hashrate — the number of hashes your CPU/GPU produces per second
- Current coin prices — cryptocurrency prices are extremely volatile and can change 50%+ in days
- Network difficulty — as more miners join a network, each miner's share decreases
- Pool fees — typically 1–2% of earned rewards
- Electricity cost — in many regions, electricity cost exceeds mining revenue for small devices
- Device uptime — a device that runs 24/7 earns proportionally more than one used part-time

Honest Expectations by Device Class

Device	Monthly Earnings Estimate	Electricity Note
Android phone (3 threads, RandomX)	~\$0.50/month at current XMR prices	Phone charger is ~5W — low cost
Raspberry Pi 4 (3 threads)	< \$1/month	5–8W power draw
Mid-range laptop (4 threads)	\$1–5/month	30–60W — check electricity cost
Desktop PC (8 threads)	\$5–20/month	100–200W — electricity may exceed earnings
High-end server (32 threads)	\$50–200/month	250–400W — viable with cheap electricity

The real strategy: Start mining on what you already own. Use the earnings calculator with your actual hashrate to see realistic projections. Reinvest mining earnings into better hardware over time. Scale up gradually — more devices means more earnings. This is a long-term, gradual income strategy, not a get-rich-quick scheme.

Always calculate your electricity cost before deciding whether mining is profitable for your setup. For Android phones and Raspberry Pis, the electricity cost is typically negligible. For desktop PCs and servers, electricity cost is a significant factor.

Universal Miner Pro ULTIMATE Edition v4.0

Built by Francois Nel — AI Technology PTY LTD — South Africa ■■

Built on a Hisense Y82 Pro. Built for everyone.

MIT Licensed — Free to use, modify, and distribute.

"Why let people suffer when any device can generate real income?"